

EX NO: 1 Reconfirm – Flag Verification via Decryption and Hash Check Walkthrough

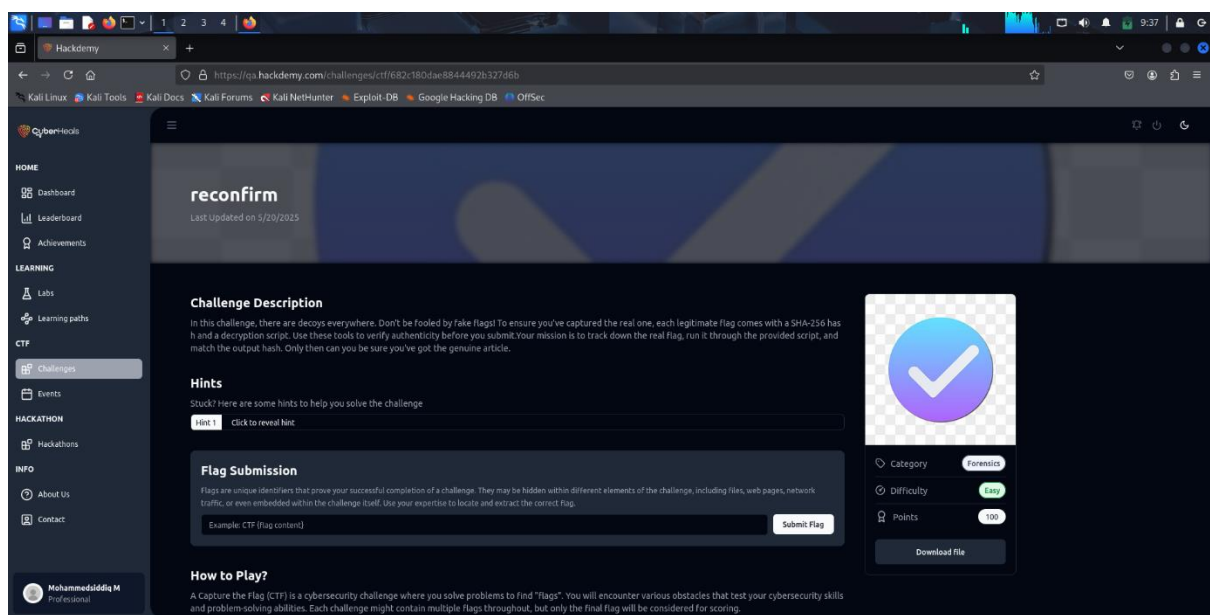
DATE:

Description:

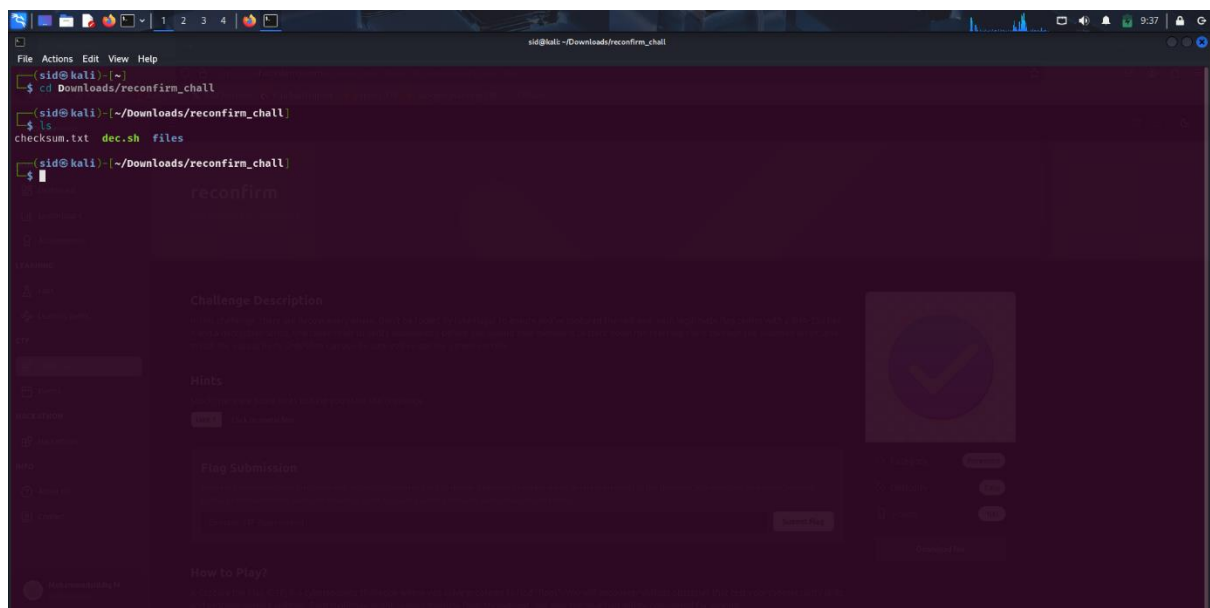
In this challenge, the goal was to identify the real flag hidden among decoys by decrypting each file using a provided script and verifying the output with a SHA-256 hash. Only the correctly decrypted flag whose hash matched the given checksum was accepted as valid.

Introduction:

The "Reconfirm" challenge tests your ability to distinguish real data from decoys by combining decryption and cryptographic hash verification. It simulates a real-world scenario where only properly decrypted and authenticated information can be trusted, reinforcing secure validation practices in cybersecurity.



Download challenge files and unzip it.



In the reconfirm_chall folder these files are available checksum.txt, dec.sh , files.

[illegible]

You can see lists of files in this folder.

```

File Actions Edit View Help
#!/bin/bash

# Check if the user provided a file name as an argument
if [ $# -eq 0 ]; then
    echo "Expected usage: dec.sh <filename>"
    exit 1
fi

# Store the provided filename in a variable
file_name="$1"

# Check if the provided argument is a file and not a folder
if [ ! -f "$file_name" ]; then
    echo "Error: '$file_name' is not a valid file. Look inside the 'files' folder with 'ls -R'"
    exit 1
fi

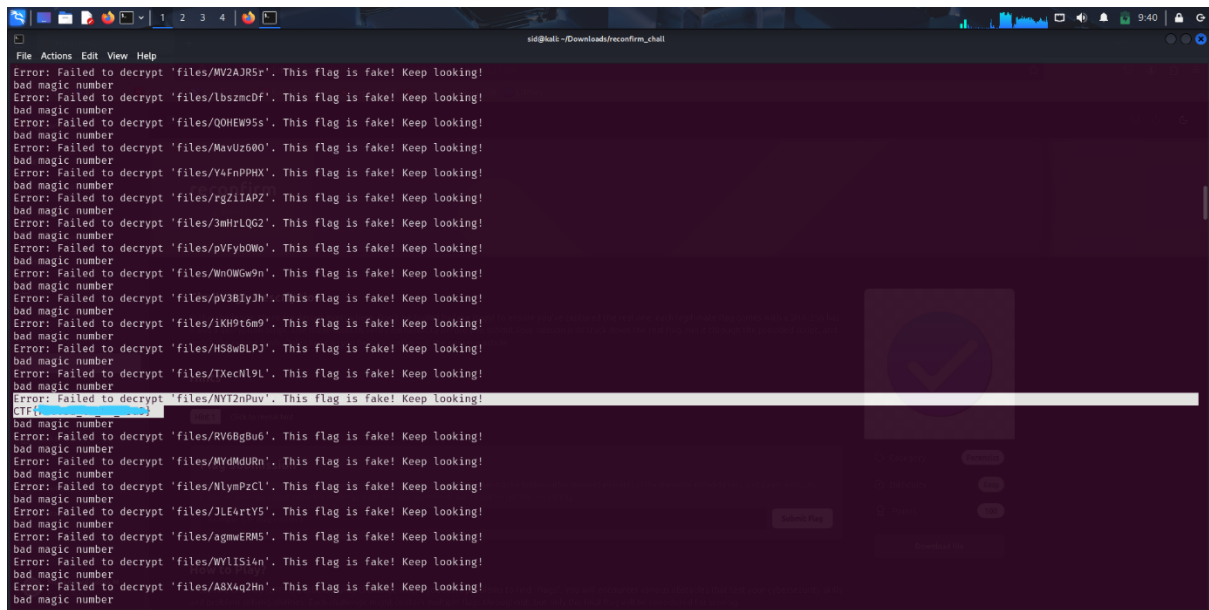
# If there's a file that is not a file, print an error message
if ! openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -salt -in "$file_name" -k trustbr34k3r; then
    echo "Error: Unable to decrypt '$file_name'. This flag is fake! Keep looking!"
    exit 1
fi

[sid@kali] ~/Downloads/reconfirm_chall
ls files
05gkMfC 3F0UCFD5 760ZV0rr 9wXkj9w8 CCh8MqFs FdH2dG8j JLuwl5UE MV2AJR5r PNNR1q3F RXQH6a3z Vc1EpYz YJae92U bnahMLHF g1qINnts lEjML22a nxx3UCp8 qK1R0SLC tmkJmhbv y0mZlQym
0aer7803 3aaAeg12 7Xk1I9M8 9wzwo1J0 CDg6f4d6 FK08M6ge JVT3kAgk MX1LXL5j PkxPOGfF RrQgXz2J Vc6s0sw2 YKYjuwGz c0io2qFy gwp0Z6a LM1AQDe oLZ1dZsM qMqCX95Y udALe5x4 yb3Ro4Y5
010i0bHl 3mH1OG2 75n1xK9w AS3J2v7V CTTDm6C3 FLEOST10 3hJzL5p MrdM0uRn PoAr70bR Rrqb3VJG VmzNFHFp ZEEL379A cYwY2TfY gPpNFLO6 lMG6fLr oNfMbvds qfHm9S0Q ch1bEVLV ypb3Ao8e
00uAY0ep 3nrx5W57 7eaPtd5 6A18G6a8 CwF6V805 GFWpJ3n6 KAS20Z1P 3d2PhwRz Q8Twtw6y ScmTy7yD WtIS14n ZQ31vMwL cZd1Ao5D mDzgkzrP ocYctY2R FXmUGw1m vQtPAQ8H 21DTGQLY
17H5i0jE 4Q8ZNSH7 7leu1S3 AUJ3jz9E Dgw0VL6K GHw0HV4M KCoDTFB7 MvDDPtV6 QTOheksH SBr9VrP6 WAA6Y2G0 ZQ7fntg7 chVzQzQY iRHf9t6m9 mWNEp8Zi ocoustR1 rZv1IAP2 wmlLoCSNS z5y1jgty
1C2HQhqe 4EqHTV18 7z3ixhLd AmnJi06E D1NWq9G6 GpbebjyX K9uXw6U MAwMCF15 QHQH9E5s TbtbYLwL JUrJfagzu ZZZeANnA cOmMk1SVH 1h4q9z1U mg4gFe01 orQXAz13 r3t4a881 vTB6VP8B yz8bAJ5R
1LPM0E7J 4qg84K00 8NFqGw6n DmWq8385 HSSABLP3 KhneTd4T NtSL5u1l QXJ3Rt8d T4x5sU1N WmHwG6wN auZ500pC d8vPbnBq jdn0hJ3c ymF2m1Zv oymMY2V rZ0K3jnc w2q0n61 20ZJR3VC
1P5d5fTb 419Wm5D5 6R0g32TY ALnLQJ1G D1MoTEV4 L9tRKUwK NYT2npuv Qm08850Q Ucbhwr0D WwobU0QA agmERM5 d1rJ3vLw kD1GMXWQ n1PE7L1z p0qWEGQ s3TrU3bC w80nVz20 zmrlJzL
2CtEUmhF 5HAM1X3J 8WA32dcd Aus0v7jR D7YK5AQz I3kD10g8 LZgD2I9b NN25jkt QmX5K83R Qdusnf5L Wx2M1r16 b5T2paRr d1rsLAWo QKML2WUP n6yqXRV8 p31Vedur 5G0AdKw8 y20sxd2Nl
26w0K3F3 5nt1krl0 6WYKh59b BR8RF4F5 Elgr895D Iw7Dp5FE LnkLtxDL NlymPc1L R84tLXGR V3xkcK07 X3z7y4Ar bbbh6FeD dZ3Zg8k0 KtDak0Ie n5epPhk6 pREKecwB sk0wKcRD whR2yGKc zP4ssY9Q
2Rld0C5 5m00074 0uMLkM9 E1CM100 7J837Ao LoS0s05t NtgmP3R RK08oCM4 V0HAKJ76 X5Vndb46 Wkch0E1V eKcEH616 kvSPf0Bw nTcEj10L v0797aTV pVfYB0W0 14t1ak1f X200hCnC
25LUe1U 62w0Wb16 9PbKw10q EYak1InX JD0ZFwH8 MAd40QmU 03clwdr4 VmSD1aA2 Y4FnPPHX bV0p3eel eow91JAR kvpk6r1P uN797aTV pVfYB0W0 14t1ak1f X200hCnC
2cdcd2de 6d2Q0040 9YFDny5 C0PNAa7J FyFLv15J J1PRVtM5 MBaThXnt OkXybbh3 Rv6B8gU6 VU1fN83 EY0owfPv bEwEJec fPxVjTBY L70c1eRr nV017uZv qAUgA8J3x tenQz1JC y2XKKK9C
2e1j3rPH 6m78P1G1 9rRtRn1R C13q9vZ FwK6S1P J1E4RTV5 MLDXw9m Ot3jkt8H M0r5Yvq Z2q0p5SM YZc8J6Vf bl5MKVCN fW6X1bf3 18t86vEL npy5Ly1P q1qCOTDP tkv0u0X3 y8TH2YHC
[sid@kali] ~/Downloads/reconfirm_chall
ls find_files -type f | while read file; do ./dec.sh "$file"; done

```

Enter command **“find files/ -type f | while read file; do ./dec.sh "\$file"; done”**. Then will

- Find every file inside files/ and subfolders
- Run dec.sh on each
- Show you the output (like "This flag is fake!" or real flag content)



Here you can find get your flag.

EX NO: 2

Primelyser – with pcapng file and wireshark Walkthrough

DATE:

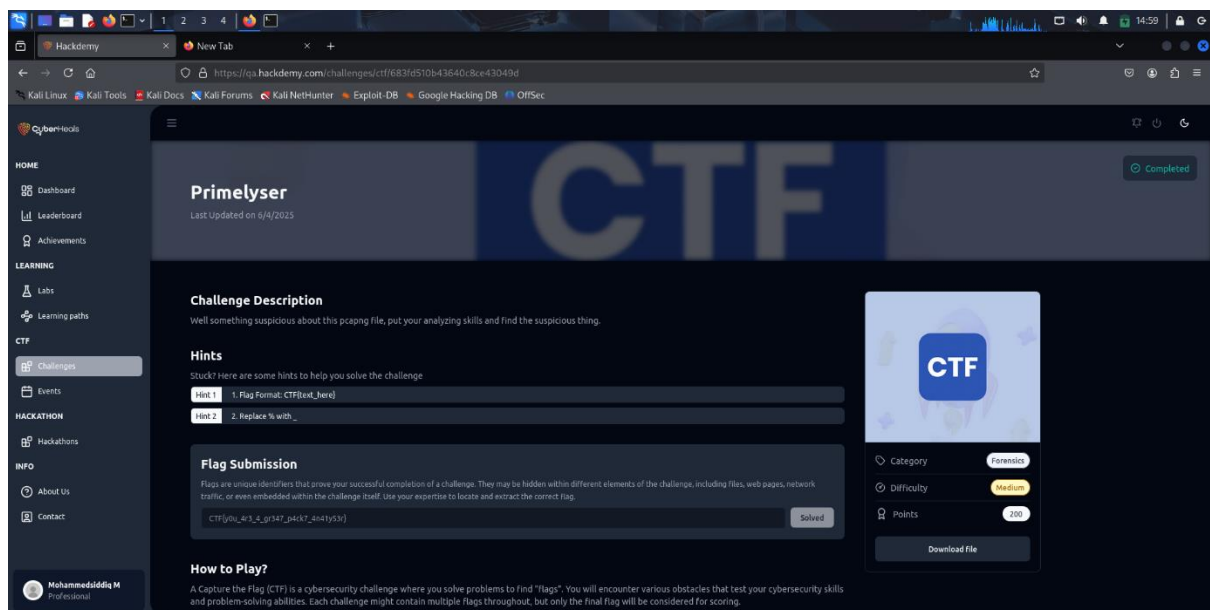
Description:

Well something suspicious about this pcapng file, put your analyzing skills and find the suspicious thing.

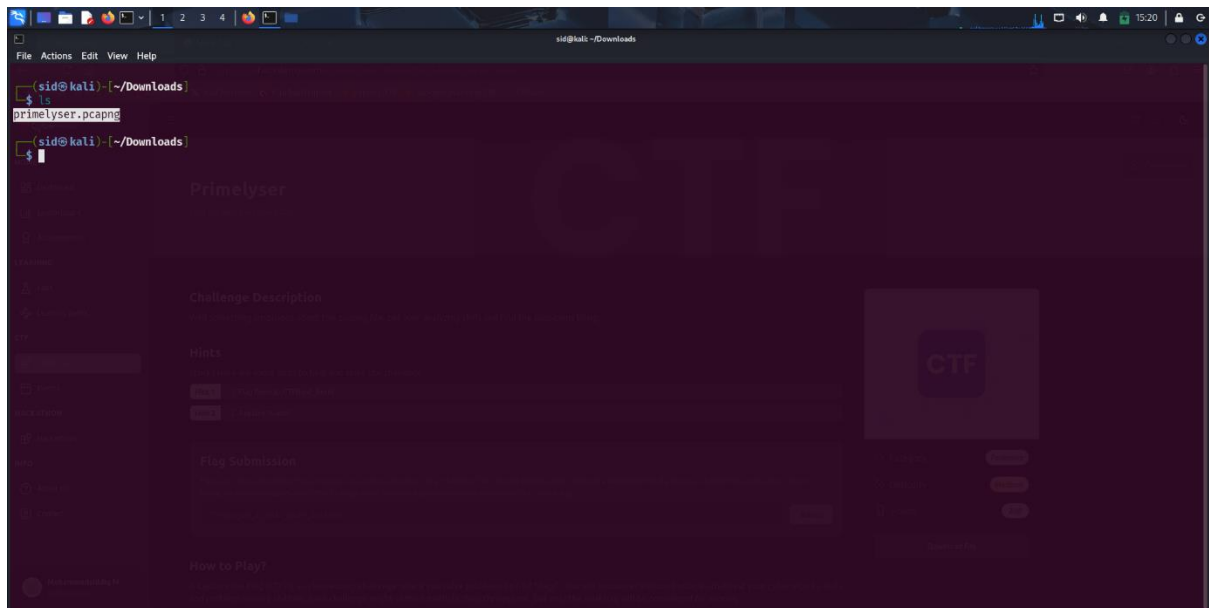
Introduction:

Wireshark is a powerful network protocol analyzer used to capture and examine packets in real-time or from saved capture files like .pcapng. It provides deep visibility into network traffic, making it an essential tool for cybersecurity analysts during forensic investigations and CTF challenges.

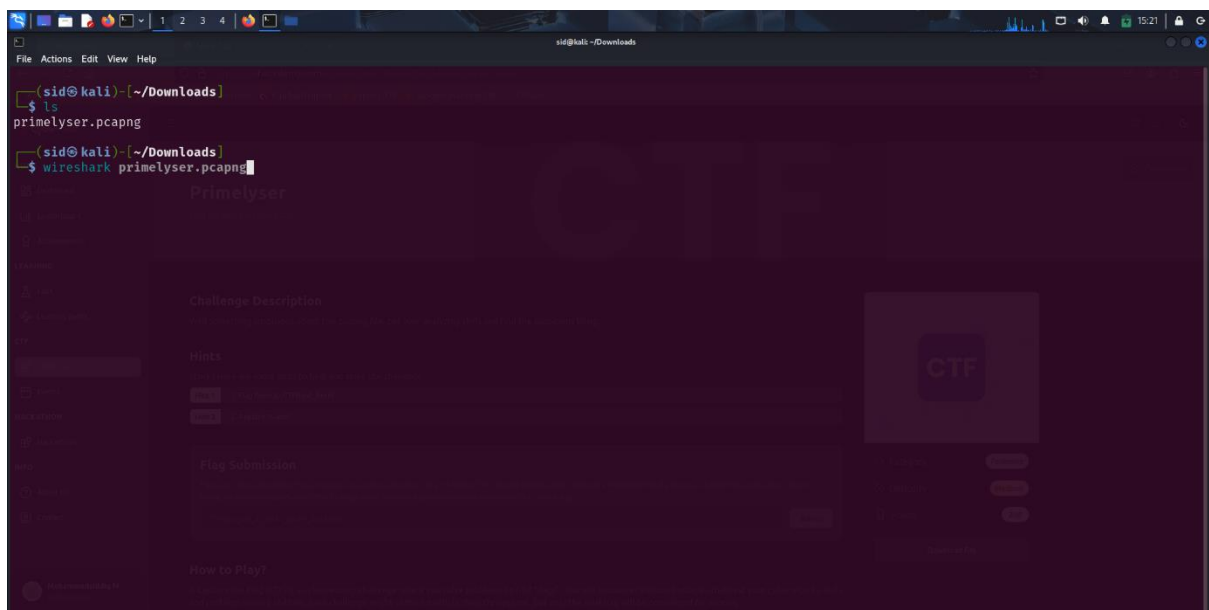
In this challenge, we analyze a .pcapng file using Wireshark to inspect captured network traffic. By carefully examining HTTP requests, DNS queries, or unusual data patterns, our goal is to uncover a hidden flag embedded within the packet capture.



Then click the Download file and it will download the .pcapng file format. Use the Wireshark Tool to view this file.



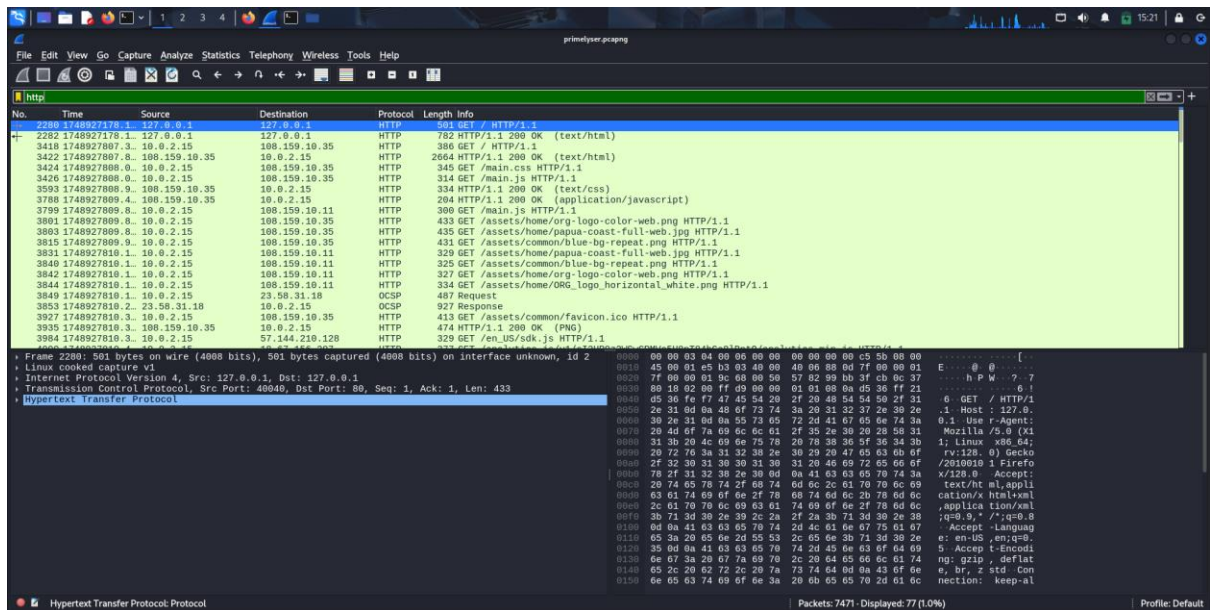
And write the command to open the file “**wireshark primelyser.pcapng**”. You can from this file in terminal or open in wireshark by clicking the option of open file.



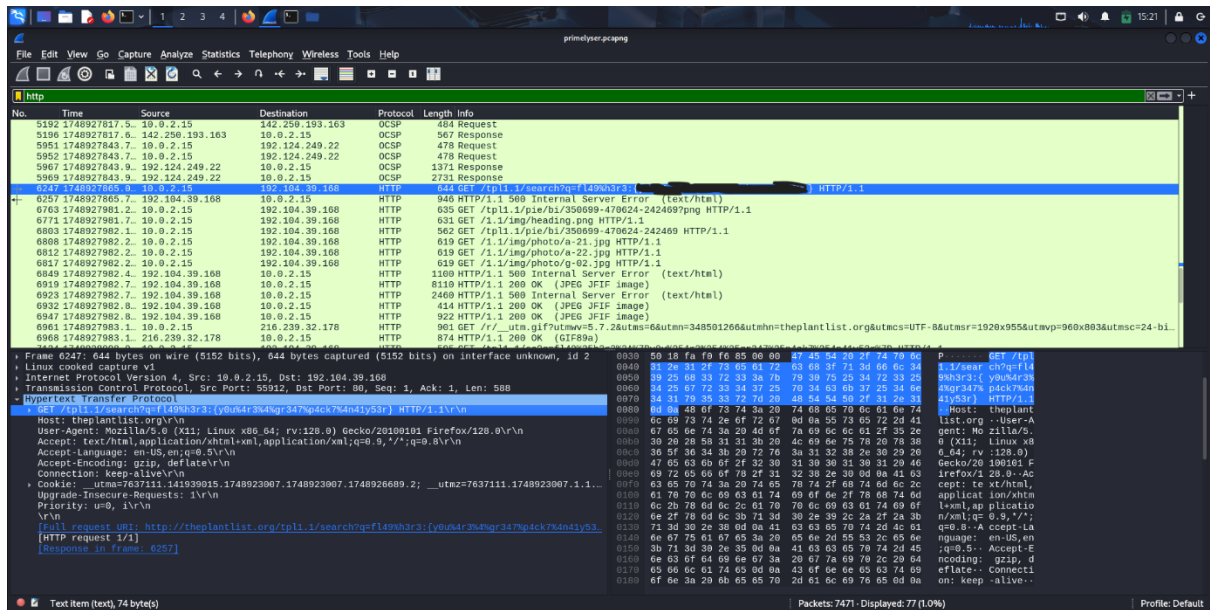
To filter HTTP traffic in Wireshark, enter the following in the display filter bar:

“http”

This will show only packets using the HTTP protocol.



Now Carefully view the each request and response by scrolling. And you will get the flag in url. After find the flag replace the % character to _ . Now submit the flag.



EX NO: 3

Key heist

DATE:

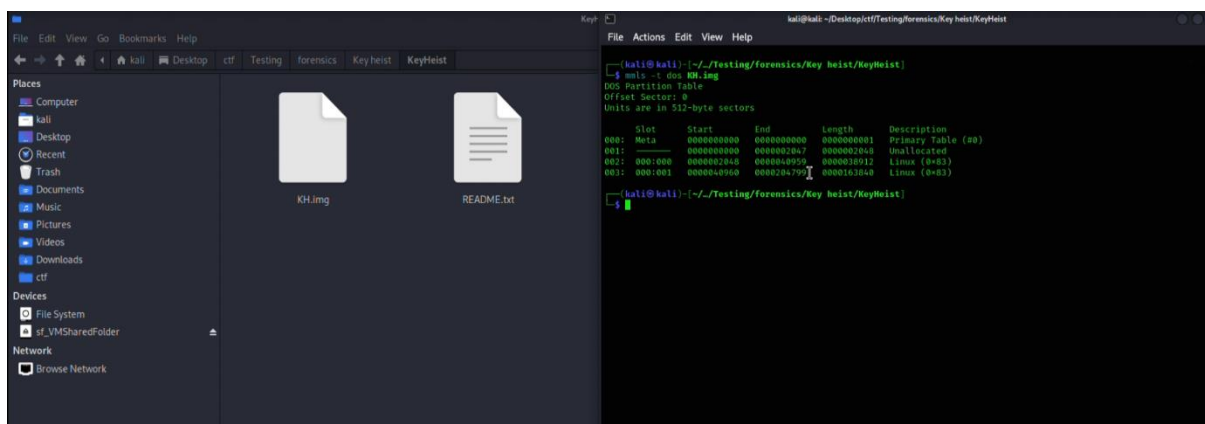
Description:

Analyze the disk image and find the hidden key. Use it to unlock the flag.

Introduction:

Then analyze file using the command “**mmls -t dos KH.img**”. **mmls** - Partition listing tool from the sleuthkit or tsk (forensics toolkit). It lists the partitions found in a disk image. **-t dos** - Specifies the partition table type. Here, dos means MBR (Master Boot Record), which is used in most legacy systems. **KH.img** - The disk image file you are analyzing. It could be a forensic image of a hard drive or removable media.

mmls -t dos KH.img lists all DOS/MBR partitions inside the KH.img file.



And use the another command to find the flag “**fls -r -o 40960 KH.img**”. It recursively lists all files and directories (including deleted ones) starting from sector 40960 in KH.img.



Use **icat** to extract files using their inode number. The command extracts the contents of inode 17 from the disk image KH.img starting at offset 40960 using **icat**. The extracted data is saved as **id_ed25519.pub**, typically for file recovery or forensic analysis.

```

(kali@kali)-[~/Testing/forensics/Key heist/KeyHeist]
$ icat -o 40960 KH.img 18 > flag.txt

(kali@kali)-[~/Testing/forensics/Key heist/KeyHeist]
$ cat flag.txt
3fef72c6412cfe71ec904cf3edde7498fa649ad9734e71b0167d356a6d7a8ac3

(kali@kali)-[~/Testing/forensics/Key heist/KeyHeist]
$ icat -o 40960 KH.img 17 > id_ed25519.pub

```

The command extracts inode 18 from the KH.img disk image at offset 40960 using icat, retrieving the raw file data. The output is saved as id_ed25519.pem, typically representing an SSH private key for authentication.

```

(kali@kali)-[~/Testing/forensics/Key heist/KeyHeist]
$ cat id_ed25519.pem
-----BEGIN RSA PRIVATE KEY-----
SGVyZSdzIHRoZSBjbml0aWZsaXphdGlvbiB2ZWNoY3I6IGVud3J5cHRpb25JbnRWZWMKSGVyZSdzIHRoZSBk6IGF1c0VuY3J5cHRpb25LZXkKS2V5
IHNpemUgaW4gQm10czogMTI4Ck1vZGU6IENCQwpJbnB1dCB1ZSh0IEZvcmlhdDogSGV4CkhpbnQ6IFVzZSBodHRwciovL3d3dy5qYXZhaW51c2UuY29t
L2Flc2dlbmVvYXRvcg==
-----END RSA PRIVATE KEY-----

(kali@kali)-[~/Testing/forensics/Key heist/KeyHeist]
$ echo 'SGVyZSdzIHRoZSBjbml0aWZsaXphdGlvbiB2ZWNoY3I6IGVud3J5cHRpb25JbnRWZWMKSGVyZSdzIHRoZSBk6IGF1c0VuY3J5cHRpb2
5LZXkKS2V5IHNpemUgaW4gQm10czogMTI4Ck1vZGU6IENCQwpJbnB1dCB1ZSh0IEZvcmlhdDogSGV4CkhpbnQ6IFVzZSBodHRwciovL3d3dy5qYXZhaW
51c2UuY29tL2Flc2dlbmVvYXRvcg==' | base64 -d
Here's the Initialization vector: encryptionIntVec
Here's the Key: aesEncryptionKey
Key size in Bits: 128
Mode: CBC
Input Text Format: Hex
Hint: Use https://www.javainuse.com/aesgenerator

(kali@kali)-[~/Testing/forensics/Key heist/KeyHeist]
$

```

Then view the captured private key and copy the key and use cyberchef for flag

The screenshot shows the CyberChef web application interface. The 'Recipe' tab is active, displaying an 'AES Decrypt' recipe. The 'Key' field is set to 'aesEncryptionKey' and the 'IV' field is set to 'encryptionIntVec'. The 'Mode' is 'CBC', 'Input' is 'Hex', and 'Output' is 'Raw'. The 'Input' field contains the base64-encoded data from the previous terminal output. The 'Output' field shows the result: 'CTF{...}'.

EX NO:4

X-Tension

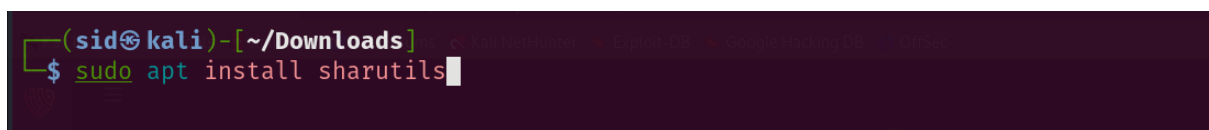
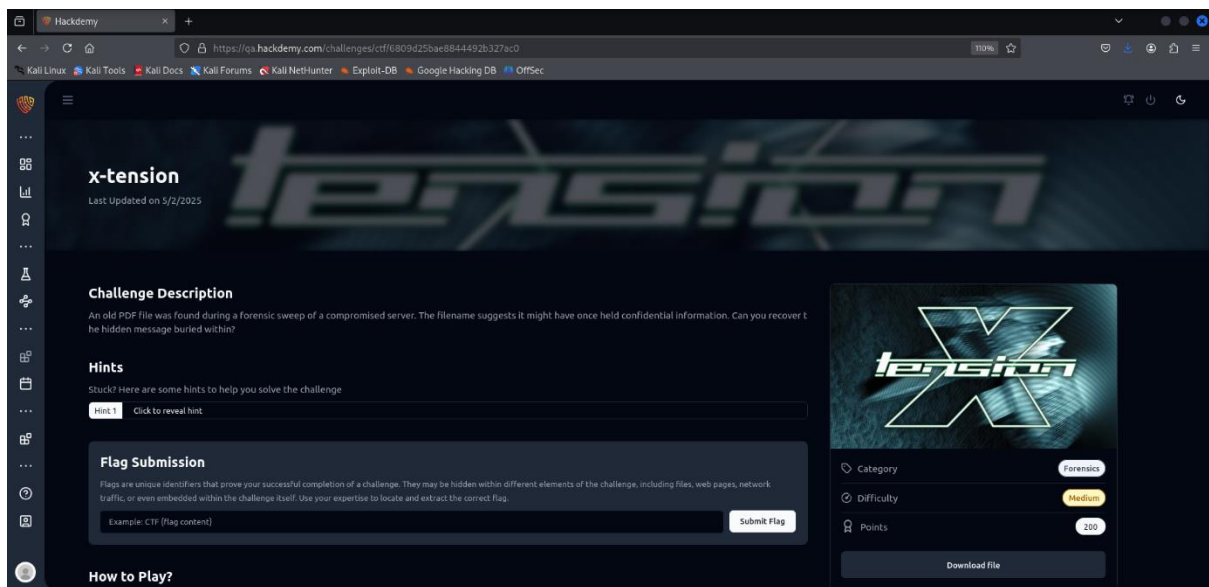
Date:

Description:

An old PDF file was found during a forensic sweep of a compromised server. The filename suggests it might have once held confidential information. Can you recover the hidden message buried within?

Introduction:

Download the challenge file from the hackdemy challenge website. Then first of all install the tool for this challenge called “sharutils”.



Check file type "file confidential.pdf". Rename the file to a shell script "mv confidential.pdf flag.sh".

```
File Actions Edit View Help
(sid@kali)~[~/Downloads]
$ ls
confidential.pdf
(sid@kali)~[~/Downloads]
$
```

```
(sid@kali)~[~/Downloads]
$ ls
confidential.pdf
(sid@kali)~[~/Downloads]
$ ls -l
total 8
-rw-rw-r-- 1 sid sid 5255 Jul 19 14:46 confidential.pdf
(sid@kali)~[~/Downloads]
$ mv confidential.pdf flag.sh
```

Make the script executable "chmod +x flag.sh". Execute the shell script "./flag.sh". This extracts archive.ar.

```
(sid@kali)~[~/Downloads]
$ mv confidential.pdf flag.sh
(sid@kali)~[~/Downloads]
$ file flag.sh
flag.sh: shell archive text
(sid@kali)~[~/Downloads]
$ ls -l
total 8
-rw-rw-r-- 1 sid sid 5255 Jul 19 14:46 flag.sh
```

```
(sid@kali)~[~/Downloads]
$ chmod +x flag.sh
(sid@kali)~[~/Downloads]
$ ./flag.sh
x - created lock directory _sh66329.
x - extracting archive.ar (text)
x - removed lock directory _sh66329.
```

Check the type of archive.ar "file archive.ar". Extract contents from .ar archive "ar x archive.ar". This reveals archive.cpio.

```
(sid@kali)-[~/Downloads]
$ ls
archive.ar  flag.sh

(sid@kali)-[~/Downloads]
$ ar x archive.ar
```

```
(sid@kali)-[~/Downloads]
$ file archive.ar
archive.ar: current ar archive

(sid@kali)-[~/Downloads]
$ ar x archive.ar

(sid@kali)-[~/Downloads]
$ ls
archive.ar  archive.cpio  flag.sh
```

Extract .cpio archive "cpio -i < archive.cpio". This gives flag.gz.bz2.

```
(sid@kali)-[~/Downloads]
$ cpio -i < archive.cpio
2 blocks

(sid@kali)-[~/Downloads]
$ ls
archive.ar  archive.cpio  flag.gz.bz2  flag.sh
```

Decompress .bz2 file "bzip2 -dv flag.gz.bz2". Output: flag.gz. Decompress .gz file "gzip -d flag.gz".

```
(sid@kali)-[~/Downloads]
$ bzip2 -dv flag.gz.bz2
flag.gz.bz2: done

(sid@kali)-[~/Downloads]
$ ls
archive.ar  archive.cpio  flag.gz  flag.sh
```

```
(sid@kali)-[~/Downloads]
$ gzip -d flag.gz

(sid@kali)-[~/Downloads]
$ ls -l
total 20
-rw-rw-r-- 1 sid sid 1092 Apr 11 02:18 archive.ar
-rw-r--r-- 1 sid sid 1024 Jul 19 14:51 archive.cpio
-rw-rw-r-- 1 sid sid 272 Jul 19 14:52 flag
-rwxrwxr-x 1 sid sid 5255 Jul 19 14:46 flag.sh
```

Check the type of flag "file flag". Rename the file "mv flag flag.lzip". And extract the file "lzip -d flag.lzip".

```

(sid@kali)-[~/Downloads]
$ file flag
flag: lzip compressed data, version: 1

(sid@kali)-[~/Downloads]
$ mv flag flag.lzip

(sid@kali)-[~/Downloads]
$ lzip -d flag.lzip

(sid@kali)-[~/Downloads]
$ ls
archive.ar  archive.cpio  flag.lzip.out  flag.sh

```

Check the type of flag.lzip.out “file flag.lzip.out”. Now rename the file “mv flag.lzip.out flag.lz4”. Extract the file flag.lz4 “lz4 -dv flag.lz4”. And now check the file type of flag “file flag”. Rename the file “mv flag flag.lzma”. Extract the file “lzma -d flag.lzma”.

```

(sid@kali)-[~/Downloads]
$ file flag.lzip.out
flag.lzip.out: LZ4 compressed data (v1.4+)

(sid@kali)-[~/Downloads]
$ mv flag.lzip.out flag.lz4

(sid@kali)-[~/Downloads]
$ lz4 -dv flag.lz4
*** lz4 v1.10.0 64-bit multithread, by Yann Collet ***
Decoding file flag
flag.lz4 : decoded 210 bytes

(sid@kali)-[~/Downloads]
$ ls
archive.ar  archive.cpio  flag  flag.lz4  flag.sh

```

```

(sid@kali)-[~/Downloads]
$ file flag
flag: LZMA compressed data, streamed

(sid@kali)-[~/Downloads]
$ mv flag flag.lzma

(sid@kali)-[~/Downloads]
$ lzma -d flag.lzma

```

Check the file type of flag and again rename the file “mv flag flag.lzop”. Extract the file flag.lzop “lzop -d flag.lzop”.

```

(sid@kali)-[~/Downloads]
$ file flag
flag: lzop compressed data - version 1.040, LZ01X-1, os: Unix

(sid@kali)-[~/Downloads]
$ mv flag flag.lzop

(sid@kali)-[~/Downloads]
$ lzop -d flag.lzop

(sid@kali)-[~/Downloads]
$ ls
archive.ar  archive.cpio  flag  flag.lz4  flag.lzop  flag.sh

```

Once again check the file type of the flag. And Rename the flag file “mv flag flag.xz”. Extract the file “xz -d flag.xz”.

```
(sid@kali)-[~/Downloads]
$ file flag
flag: XZ compressed data, checksum CRC64

(sid@kali)-[~/Downloads]
$ mv flag flag.xz

(sid@kali)-[~/Downloads]
$ xz -d flag.xz

(sid@kali)-[~/Downloads]
$ ls
archive.ar  archive.cpio  flag  flag.lz4  flag.lzop  flag.sh
```

Finally you can view the file type of the flag file. Then view the encrypted flag. Decrypt it using “cat flag | xxd -r -p”.

```
(sid@kali)-[~/Downloads]
$ file flag
flag: ASCII text

(sid@kali)-[~/Downloads]
$ cat flag
4354467b7033346b5f347263683176335f7033726630726d346e63337d

(sid@kali)-[~/Downloads]
$ cat flag | xxd -r -p
CTF{XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX}
```

EX NO:5

Sleuthkit 1

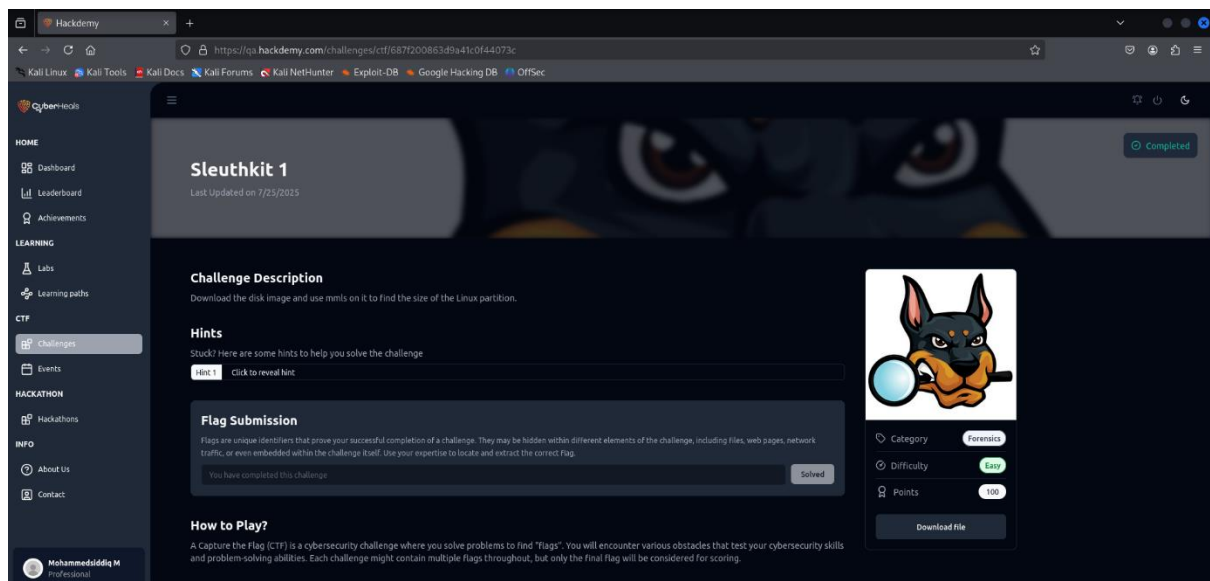
DATE:

Description:

Download the disk image and use mmls on it to find the size of the Linux partition.

Introduction:

Download the disk image and use mmls on it to find the size of the Linux partition.
Download the challenge files like .img, .py both files.



Open the terminal in that file location.

```
(sid@kali)-[~/Downloads/sleuthkit1]
$ ls
sleuthkit_1.img  sleuthkit_chall.py

(sid@kali)-[~/Downloads/sleuthkit1]
$
```

And run the .py file in the terminal by the command python3 filename.py.

```
(sid@kali)-[~/Downloads/sleuthkit1]
$ python3 sleuthkit_chall.py
===== Sleuthkit Intro Challenge =====
Use mmls to find the Linux partition size of the disk image.
Enter the Linux partition size: 
```

When you run the python file you need size of image file to access flag. Then open the new terminal and view the .img file details by the command “file filename.img”.

```
(sid@kali)-[~/Downloads/sleuthkit1]
$ file sleuthkit_1.img
sleuthkit_1.img: DOS/MBR boot sector; partition 1 : ID=0x83, start-CHS (0x0,3,5), end-CHS (0x14,2,8), startsector 100, 2532 sectors
```

The mmls command is used in digital forensics to list the partition layout of a disk image. It comes from the Sleuth Kit (TSK) suite. use the command to analyze the image file “mmls filename.img”.

```
(sid@kali)-[~/Downloads/sleuthkit1]
$ mmls sleuthkit_1.img
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors
```

Partition	Slot	Start	End	Length	Description
000:	Meta	0000000000	0000000000	0000000001	Primary Table (#0)
001:		0000000000	0000000099	0000000100	Unallocated
002:	000:000	0000000100	0000002631	0000002532	Linux (0x83)
003:		0000002632	0000020479	0000017848	Unallocated

This command analyzes the disk.img and displays information about its Partition type, Start sector, End sector, Length (number of sectors), Description (e.g., NTFS, Linux, etc.). It will show all details. Then you want copy the size of the image file.

```
(sid@kali)-[~/Downloads/sleuthkit1]
$ python3 sleuthkit_chall.py
== Sleuthkit Intro Challenge ==
Use mmls to find the Linux partition size of the disk image.
Enter the Linux partition size: 0000002532
```

Hit the enter button you will get the flag.

```
(sid@kali)-[~/Downloads/sleuthkit1]
$ python3 sleuthkit_chall.py
== Sleuthkit Intro Challenge ==
Use mmls to find the Linux partition size of the disk image.
Enter the Linux partition size: 0000002532

Correct! Here is your flag:
CTF{[REDACTED]} challenge Description
```

EX NO:6

Sleuthkit2

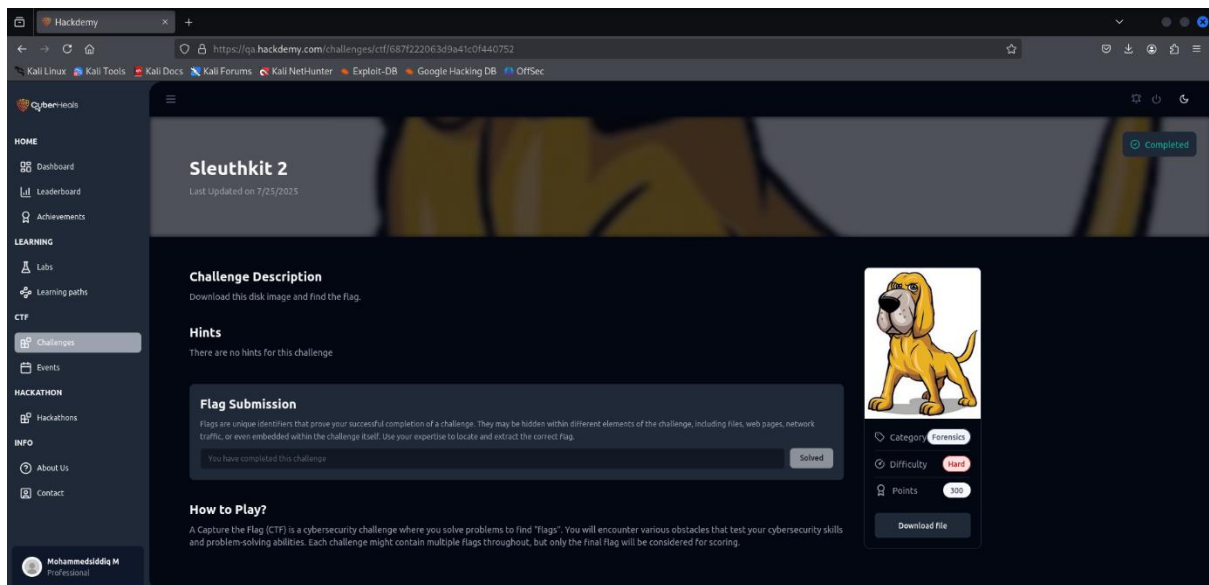
DATE:

Description:

Download this disk image and find the flag.

Introduction:

Download the image file.



Analyzes the image file by using command “file filename.img”.

```
(sid@kali) - [~/Downloads/sleuthkit2]
$ file sleuthkit_2.img
sleuthkit_2.img: DOS/MBR boot sector; partition 1 : ID=0x83, start-CHS (0x0,32,33), end-CHS (0xc,190,50), startsector 2048, 202752 sectors, extended partition table (last)
```

The mmls command is used in digital forensics to list the partition layout of a disk image. It comes from the Sleuth Kit (TSK) suite. Use the command to analyze the image file “mmls filename.img”.

```
(sid@kali) - [~/Downloads/sleuthkit2]
$ mmls sleuthkit_2.img
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

Slot      Start      End      Length    Description
000:  Meta      0000000000 0000000000 0000000001 Primary Table (#0)
001:  _____ 0000000000 0000002047 0000002048 Unallocated
002:  000:000 0000002048 0000204799 0000202752 Linux (0x83)
```

Then open the autopsy tool, Click on the New case. And fill the case name like case 1 or anyother challenge name. Leave the description blank, its not to fill. Write investigator name like test1 or something.

The screenshot shows a web form titled "CREATE A NEW CASE" on a light green background. The form is divided into three sections: 1. **Case Name:** A text input field containing "case1". 2. **Description:** A single-line text input field that is empty. 3. **Investigator Names:** A grid of ten text input fields labeled a. through j. Field 'a.' contains "test1", while fields b. through j. are empty. At the bottom of the form are three buttons: "NEW CASE", "CANCEL", and "HELP".

Create a new case for the testing image file. Select the name of the investigator and click on add the host.

The screenshot shows a confirmation screen titled "Creating Case: case1". It displays two lines of text: "Case directory (/var/lib/autopsy/case1/) created" and "Configuration file (/var/lib/autopsy/case1/case.aut) created". Below this, it says "We must now create a host for this case." and "Please select your name from the list:" followed by a dropdown menu showing "test1" with a downward arrow. At the bottom is a large orange button labeled "ADD HOST".

Write the host name on the column. Scroll down and click on the add host option.

The screenshot shows a web form titled "ADD A NEW HOST" on a light green background. The form is divided into four sections: 1. **Host Name:** A text input field containing "host1". 2. **Description:** A single-line text input field that is empty. 3. **Time zone:** A text input field that is empty. 4. **Timeskew Adjustment:** A text input field that is empty. At the bottom of the form is a large orange button labeled "ADD HOST".

And click on the add image file option in the autopsy. Now paste the file path of the image file.

Adding host: host1 to case case1

Host Directory (/var/lib/autopsy/case1/host1/) created

Configuration file (/var/lib/autopsy/case1/host1/host.aut) created

We must now import an image file for this host

ADD IMAGE

Case: case1
Host: host1

ADD A NEW IMAGE

1. **Location**
Enter the full path (starting with /) to the image file.
If the image is split (either raw or EnCase), then enter '*' for the extension.

/home/sid/Downloads/sleuthkit2/sleuthkit_2.img

2. **Type**
Please select if this image file is for a disk or a single partition.

☒ Disk ☐ Partition

3. **Import Method**
To analyze the image file, it must be located in the evidence locker. It can be imported from its current location using a symbolic link, by copying it, or by moving it. Note that if a system failure occurs during the move, then the image could become corrupt.

☒ Symlink ☐ Copy ☐ Move

NEXT

CANCEL HELP

Don't click on anyother option below. Just move to next and once again click on the add options.

Image File Details

Local Name: images/sleuthkit_2.img

Data Integrity: An MD5 hash can be used to verify the integrity of the image. (With split images, this hash is for the full image file)

☒ Ignore the hash value for this image.
☐ Calculate the hash value for this image.
☐ Add the following MD5 hash value for this image:

☐ Verify hash after importing?

File System Details

Analysis of the image file shows the following partitions:

Partition 1 (Type: Linux (0x83))
Sector Range: 2048 to 204799
Mount Point: /1/ File System Type: ext

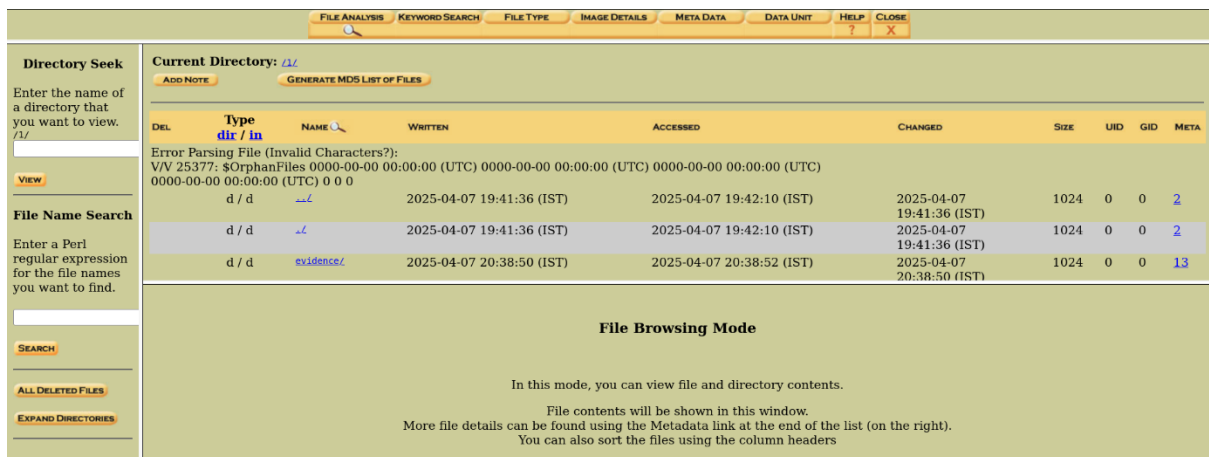
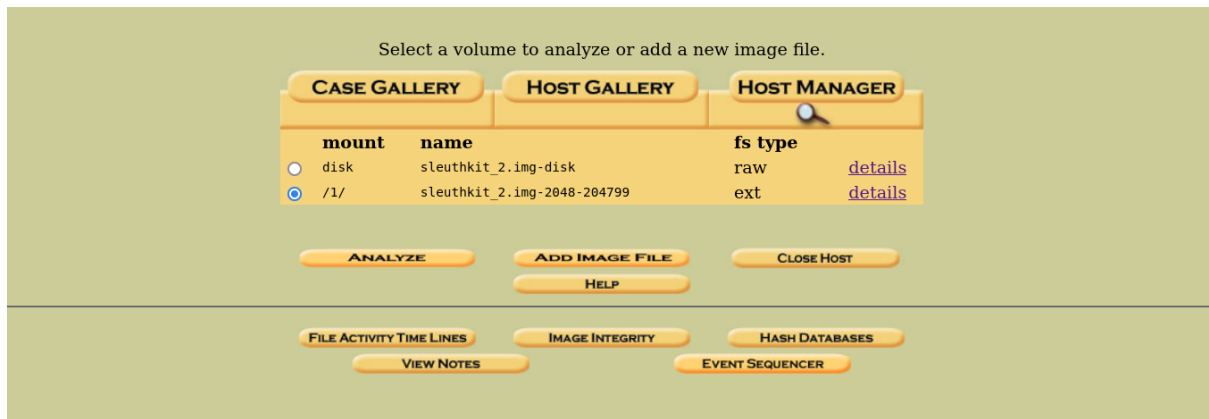
ADD CANCEL HELP

For your reference, the mmls output was the following:

DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

Slot	Start	End	Length	Description
002:	000:000	0000002048	0000204799	0000202752 Linux (0x83)

Then click on ok button to start the process. Click on the analyze option, Next step click on file analyze option it will show all finding using autopsy.



Click on the image details to view all details of the image file. Finally click on keyword search options to search format of flag like CTF.



Then click on shown result and now you can find flag for this challenge. Successfully captured the flag and finished the challenge.

EX NO:7

Sleuthkit 3

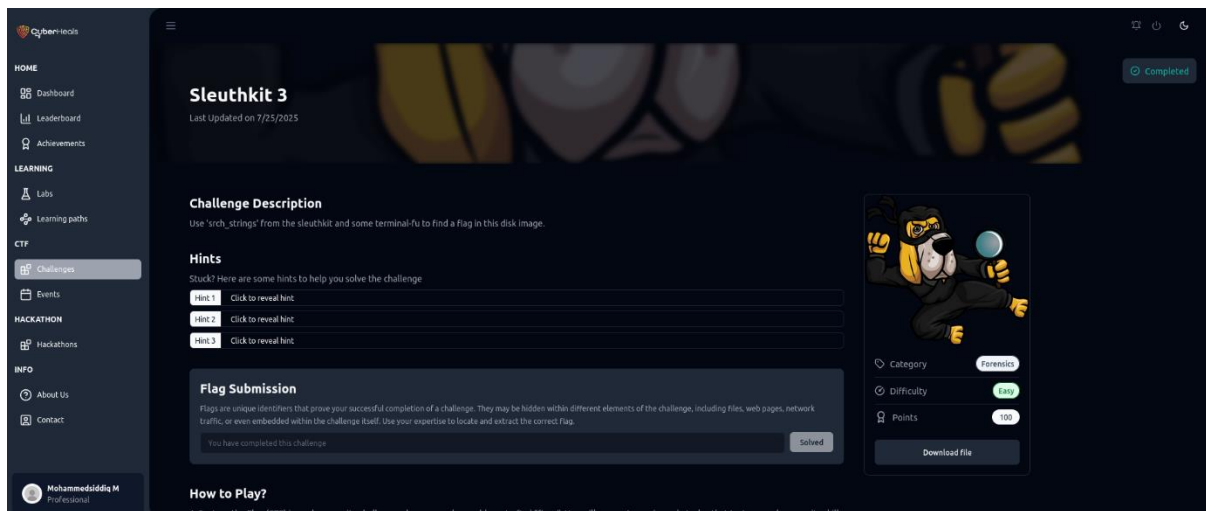
DATE:

Description:

Use 'srch_strings' from the sleuthkit and some terminal-fu to find a flag in this disk image.

Introduction:

Download the image file.



Analyzes the image file by using command “file filename.img”.

```
(sid@kali) ~/Downloads/sleuthkit3
$ ls
sleuthkit_3.img

(sid@kali) ~/Downloads/sleuthkit3
$ file sleuthkit_3.img
sleuthkit_3.img: DOS/MBR boot sector; partition 1 : ID=0x83, start=CHS (0x10,0,1), end=CHS (0x3ff,3,32), startsector 2048, 129024 sectors
```

The strings command in Linux is used to extract readable text strings from binary files, such as executables, images, or core dumps. It's especially useful in reverse engineering, forensics, and malware analysis to uncover hidden messages or clues.

Use the String command to extract the flag from the image.

```
(sid@kali)-[~/Downloads/sleuthkit3]
$ strings sleuthkit_3.img
/home/kali/disk_mount
3tZp
+r<i
lost+found
data
misc
logs
flag.txtLorem
FLAG.TXT
TheFakeOne.txt
TheRealOne.txt
FLAG.txt
hidden.txt
secret.txt
attacklogs.txt
CTF.txt
miscellaneous.txt
onepiece.txt
Vivamus lacinia odio vitae vestibulum vestibulum.
Suspendisse potenti.Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Vivamus lacinia odio vitae vestibulum vestibulum.
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the
1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but
also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets
containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum.Lorem
```

The grep command in Linux is used to search for patterns (typically text strings or regular expressions) within files or output. It's one of the most powerful and widely used command-line tools for text searching.

We can use the both command to extract the flag from the image file.

```
(sid@kali)-[~/Downloads/sleuthkit3]
$ strings sleuthkit_3.img | grep 'CTF'
CTF.txt
CTF{[REDACTED]}
CTF.txt
CTF.txt
CTF.txt
```

Successfully captured the flag and finished the challenge.

EX NO: 8

Double Vision

DATE:

Description:

The Network Operations Center (NOC) of your local institution picked up a suspicious file, they're getting conflicting information on what type of file it is. They've brought you in as an external expert to examine the file. Can you extract all the information from this strange file?

Introduction:

Download the challenge file. Try to view the file, but it is not in human readable format.

```
(sid@kali)~[/Downloads/double_vision]
$ ls
DoubleVision.txt

(sid@kali)~[/Downloads/double_vision]
$ cat DoubleVision.txt
•PNG
||
IHDR•dw9~c cHRMz6•++++•u0+`•p++Q<bKGd•++++ X•+tIME•
•JeN!•+•$!•+4•Y•d•D•(I• EIu+++{+++==++Z+/+++•++++t•+^{}o+++++b
                                     +•t+k•||•+++0,+!`0

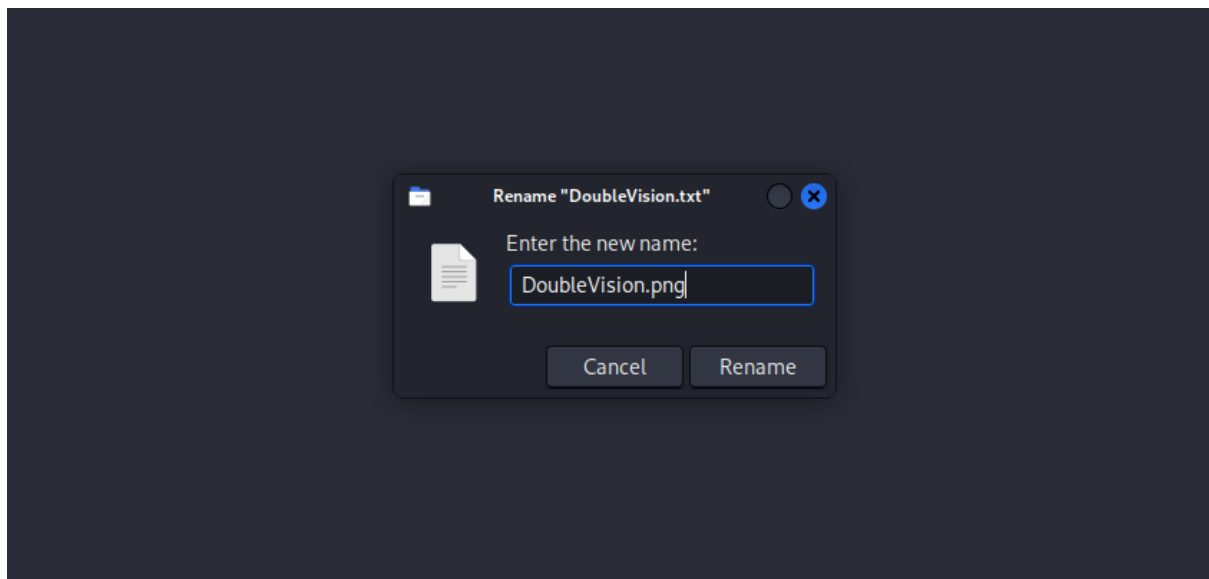
Challenge Description
C•`||•+•||•+++0,+!`0

Hints
C•`||•+•||•+++0,+!`0
```

Analyzes the image file by using command “file filename.img”.

```
(sid@kali)-[~/Downloads/double_vision]
$ file DoubleVision.txt
DoubleVision.txt: PNG image data, 400 x 100, 16-bit/color RGB, non-interlaced
```

The challenge file is text file format try to change it in .png format.



And the half of flag is now visible in .png file.

CTF{

exiftool is a powerful command-line utility used to read, write, and edit metadata in files—especially images, videos, and documents. It's often used in digital forensics, OSINT, photography, and general file inspection.

Try to use this tool and view the metadata of the .png file.

```
(sid@kali) - [~/Downloads/double_vision]
$ exiftool DoubleVision.png
ExifTool Version Number      : 13.25
File Name                    : DoubleVision.png
Directory                   : .
File Size                   : 32 kB
File Modification Date/Time  : 2025:07:28 21:45:27+05:30
File Access Date/Time       : 2025:07:28 21:51:02+05:30
File Inode Change Date/Time  : 2025:07:28 21:51:02+05:30
File Permissions             : -rw-rw-r--
File Type                   : PNG
File Type Extension         : png
MIME Type                   : image/png
Image Width                 : 400
Image Height                : 100
Bit Depth                   : 16
Color Type                  : RGB
Compression                 : Deflate/Inflate
Filter                      : Adaptive
Interlace                   : Noninterlaced
White Point X               : 0.3127
White Point Y               : 0.329
Red X                       : 0.64
Red Y                       : 0.33
Green X                     : 0.3
Green Y                     : 0.6
Blue X                      : 0.15
Blue Y                     : 0.06
Background Color            : 65535 65535 65535
Modify Date                 : 2025:04:25 10:34:48
Warning                     : [minor] Trailer data after PNG IEND chunk
Image Size                  : 400x100
Megapixels                  : 0.040
```

Analyzes the each info in that metadata and notice the warning message.

ghex (GNOME Hex Editor) is a graphical hex editor used to inspect and edit binary files at the byte level. It is especially useful in Digital forensics, Reverse engineering, File format analysis, Malware analysis, CTFs (Capture the Flag) competitions.

Open the file with help of this tool. Search for the hint you got in the metadata.



Finally open the extracted pdf and view the flag. First half of the flag from that image, second half of the flag in the .pdf file.

.pptm

Description:

I've hidden a flag in this file. Can you find it?

Download the challenge file from the site. Open and view the content inside of that files. Open the terminal where the file downloaded in that path.

```
(sid@kali)-[~/Downloads/.pptm]
$ ls
challenge.pptm
```

Then change the file format .pptm to .zip file by the command “mv challenge.pptm challenge.zip.

```
(sid@kali)-[~/Downloads/.pptm]
$ mv challenge.pptm challenge.zip

(sid@kali)-[~/Downloads/.pptm]
$ ls
challenge.zip
```

Unzip the zipped file and search for the flag file, its is in numbers.

```
(sid@kali) ~/Downloads/.pptm
$ unzip challenge.zip
Archive:  challenge.zip
  inflating: [Content_Types].xml
    creating: _rels/
  inflating: _rels/.rels
    creating: ppt/
    creating: ppt/challenge Description
    creating: ppt/slides/
  inflating: ppt/slides/slide1.xml
  inflating: ppt/slides/slide3.xml
  inflating: ppt/slides/slide2.xml
    creating: ppt/slides/_rels/
  inflating: ppt/slides/_rels/slide2.xml.rels
  inflating: ppt/slides/_rels/slide1.xml.rels
  inflating: ppt/slides/_rels/slide4.xml.rels
  inflating: ppt/slides/_rels/slide3.xml.rels
  inflating: ppt/slides/slide4.xml
  inflating: ppt/presentation.xml
    creating: ppt/theme/
  inflating: ppt/theme/theme2.xml
  inflating: ppt/theme/theme1.xml
    creating: ppt/slides/avouts/...
```

Copy the encrypted numeric values in that file.

```
(sid@kali)~[~/Downloads/.pptm/ppt]Description
$ cd slideLayouts

(sid@kali)~[~/Downloads/.pptm/ppt/slideLayouts]
$ ls
_rels  slideLayout1.xml  slideLayout2.xml  slideLayout3.xml  slideLayout4.xml  slideLayout5.xml

(sid@kali)~[~/Downloads/.pptm/ppt/slideLayouts]
$ cd ..

(sid@kali)~[~/Downloads/.pptm/ppt]mission
$ cd slideMasters

(sid@kali)~[~/Downloads/.pptm/ppt/slideMasters]
$ ls
_rels  hidden  slideMaster1.xml  slideMaster4.xml

(sid@kali)~[~/Downloads/.pptm/ppt/slideMasters]
$ cat hidden
43 54 46 7b 44 31 64 5f 75 5f 6b 6e 30 77 5f 70 74 73 5f 72 5f 7a 31 70 35 7d
```

Paste it in the cyberchef to decrypt the flag.

Download CyberChef

Last built: 5 days ago - Version 10 is here! Read about the new features here

Options About / Support

Operations

Recipe

Input

Search...

Favourites

To Base64

From Base64

To Hex

From Hex

To Hexdump

From Hexdump

URL Decode

Regular expression

Entropy

Fork

Magic

Data format

Encryption / Encoding

Public Key

Arithmetic / Logic

Networking

STEP

BAKE!

Auto Bake

43 54 46 7b 44 31 64 5f 75 5f 6b 6e 38 77 5f 78 79 74 73 5f 72 5f 7a 31 76 35 7d

From Hex will produce
"CTF{011 0 3n0w_p01ts...
r_z1p9}"

Output

43 54 46 7b 44 31 64 5f 75 5f 6b 6e 38 77 5f 78 79 74 73 5f 72 5f 7a 31 76 35 7d

Download CyberChef

Last built: 5 days ago - Version 10 is here! Read about the new features here

Options About / Support

Operations

Recipe

Input

Search...

Favourites

To Base64

From Base64

To Hex

From Hex

To Hexdump

From Hexdump

URL Decode

Regular expression

Entropy

Fork

Magic

Data format

Encryption / Encoding

Public Key

Arithmetic / Logic

Networking

STEP

BAKE!

Auto Bake

43 54 46 7b 44 31 64 5f 75 5f 6b 6e 38 77 5f 78 79 74 73 5f 72 5f 7a 31 76 35 7d

From Hex

Delimiter
Space

Output

CTF{011 0 3n0w_p01ts...
r_z1p9}

Successfully captured the flag and finished the challenge.

EX NO:10

Whispering Pixels

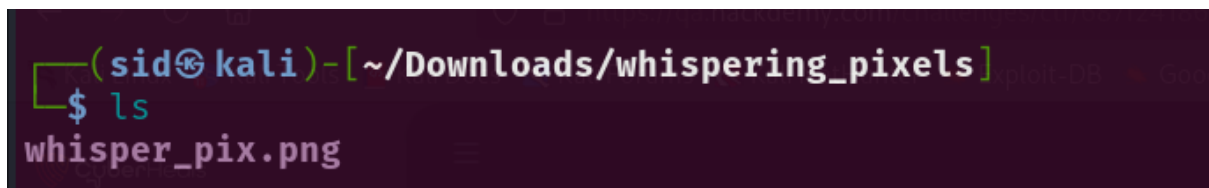
DATE:

Description:

Download this image and find the flag.

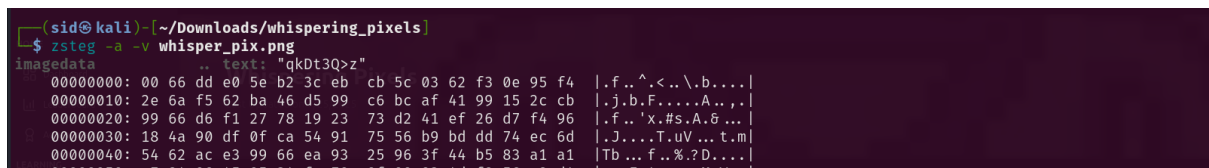
Introduction:

Download the challenge .png file from the site.

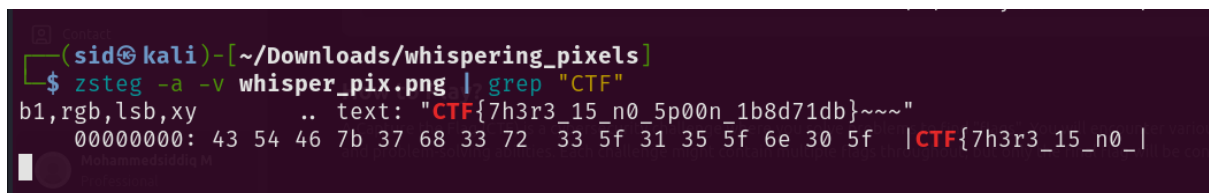


zsteg analyzes hidden messages in image files by checking different color channels (red, green, blue, alpha) and bit planes (LSB, MSB).

Run the command “zsteg -a -v chall.png”.



On bit plane 1, red channel, MSB (most significant bit), using ASCII decoding → found: Hello there!



Successfully captured the flag and finished the challenge.

EX NO:11

Hash and Apply

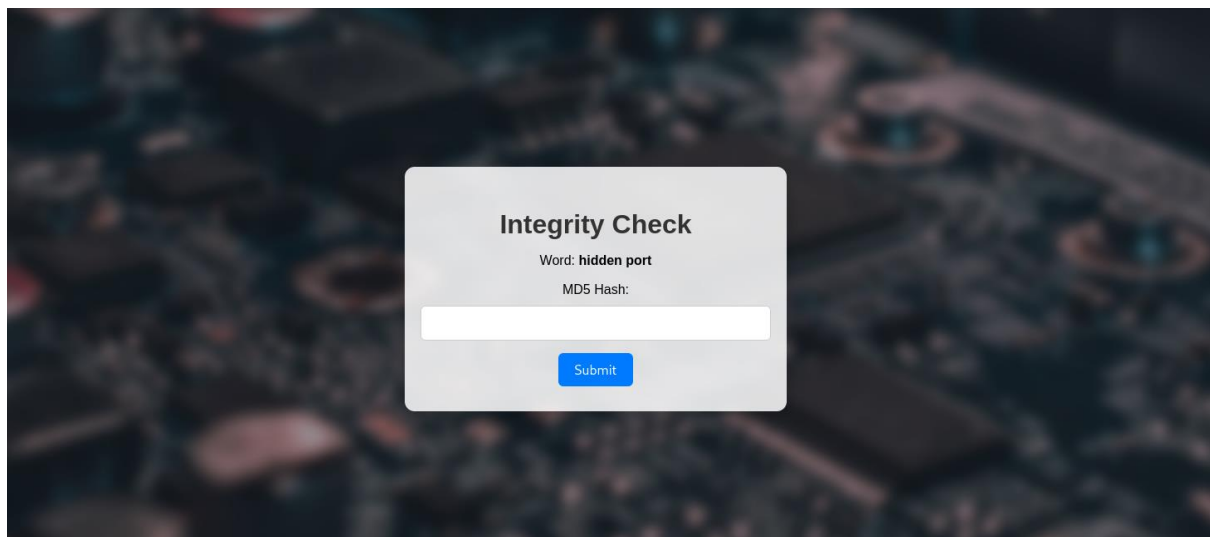
DATE:

Description:

No collisions, just decisions check the hash!

Introduction:

Access the challenge by click on the access.



copy word give in this challenge. Paste in into the online md5 hash convertor.

MD5 Hash Generator

• [Sha1 Hash Generator](#)

Use this generator to create an MD5 hash of a string:

hidden port

Generate →

Your String	hidden port	
MD5 Hash	5b6e6f524ffe7bca90353b1d201279fd	<button>Copy</button>
SHA1 Hash	1de79edb46b7ea517ed7bc0cbc84dd4b1eb82c1f	<button>Copy</button>

Perform the same step repeatedly like 3 to 5 times.



Congrats! You checked the integrity.

You got 3 out of 3 correct!

CTF{~~W41T3R_0N_0_1N6~~}

Play Again